

AI Application Lecture 6

Probabilistic Language Models and Sampling

SUZUKI, Atsushi^a

^aI thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
4. How do we determine the output token from a distribution? - Sampling and beam search
5. Can we adapt the generation to real application preference? - Hyperparameters
6. Summary
7. Next Time

Review of the previous lecture

In the previous lecture, we discussed the **pipeline** concept, where a neural network is combined with other algorithms.

Review of the previous lecture

In the previous lecture, we discussed the **pipeline** concept, where a neural network is combined with other algorithms.

- We learned that **tokenization** is the crucial entry and exit point for natural language processing pipelines, converting between raw text strings and sequences of tokens.

Review of the previous lecture

In the previous lecture, we discussed the **pipeline** concept, where a neural network is combined with other algorithms.

- We learned that **tokenization** is the crucial entry and exit point for natural language processing pipelines, converting between raw text strings and sequences of tokens.
- We examined Byte Pair Encoding (BPE), understanding its motivation: balancing sequence length reduction against the need to handle unknown words.

From tokens to probabilities

Building on what we've learned, today we address the next logical step.

From tokens to probabilities

Building on what we've learned, today we address the next logical step.

- **Premise:** We now know how to convert text to and from token sequences.

From tokens to probabilities

Building on what we've learned, today we address the next logical step.

- **Premise:** We now know how to convert text to and from token sequences.
- **Question:** How do we make a neural network generate a probable sequence of tokens?

From tokens to probabilities

Building on what we've learned, today we address the next logical step.

- **Premise:** We now know how to convert text to and from token sequences.
- **Question:** How do we make a neural network generate a probable sequence of tokens?

A neural network is a continuous function, but a token sequence is a discrete object.

From tokens to probabilities

Building on what we've learned, today we address the next logical step.

- **Premise:** We now know how to convert text to and from token sequences.
- **Question:** How do we make a neural network generate a probable sequence of tokens?

A neural network is a continuous function, but a token sequence is a discrete object.

We will have the neural network represent a **probabilistic language model** and use that model along with a pseudo-random number generator to build a **token generator**.

Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Understand the framework of viewing a continuous neural network as a **probabilistic language model** by using softmax normalization to obtain a probability mass function.

Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Understand the framework of viewing a continuous neural network as a **probabilistic language model** by using softmax normalization to obtain a probability mass function.
- Recursively construct a **token generator** from a probabilistic language model and a pseudo-random number sequence.

Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Understand the framework of viewing a continuous neural network as a **probabilistic language model** by using softmax normalization to obtain a probability mass function.
- Recursively construct a **token generator** from a probabilistic language model and a pseudo-random number sequence.
- Rigorously define and generate token sequences using various samplers, including greedy search, beam search, and methods involving temperature, top- k , top- p , and repetition penalty.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How do we use a NN in discrete token generations? - Probabilistic language model

4. How do we determine the output token from a distribution? - Sampling and beam search

5. Can we adapt the generation to real application preference? - Hyperparameters

6. Summary

7. Next Time

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How do we use a NN in discrete token generations? - Probabilistic language model

4. How do we determine the output token from a distribution? - Sampling and beam search

5. Can we adapt the generation to real application preference? - Hyperparameters

6. Summary

7. Next Time

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} := \{1, \dots, k\}$

Function:

- $f : \mathcal{X} \rightarrow \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x .

Definition:

- (LHS) $:=$ (RHS): Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\mathbf{a} = (a_1, a_2, \dots)$.

- A function $\mathbf{a} : [1, n]_{\mathbb{Z}} \rightarrow \mathcal{A}$.
- Length is denoted by $|\mathbf{a}|$.

Vector: Denoted by $\mathbf{v}$.

- A column of numbers, $\mathbf{v} \in \mathbb{R}^n$.
- i -th element is v_i .

Matrix: Denoted by $\mathbf{A}$.

- $m \times n$ matrix: $\mathbf{A} \in \mathbb{R}^{m,n}$.
- (i, j) -th element is $a_{i,j}$.
- Transpose: $\mathbf{A}^{\top}$.

Tensor: Denoted by $\underline{\mathbf{A}}$.

- Simply a multi-dimensional array.
- Vector $\rightarrow$ 1st-order, Matrix $\rightarrow$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How do we use a NN in discrete token generations? - Probabilistic language model

4. How do we determine the output token from a distribution? - Sampling and beam search

5. Can we adapt the generation to real application preference? - Hyperparameters

6. Summary

7. Next Time

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How do we use a NN in discrete token generations? - Probabilistic language model

4. How do we determine the output token from a distribution? - Sampling and beam search

5. Can we adapt the generation to real application preference? - Hyperparameters

6. Summary

7. Next Time

Question: A neural network is a continuous function. In natural language generation, we want to output a one-hot vector sequence.

How

Question: A neural network is a continuous function. In natural language generation, we want to output a one-hot vector sequence.

How do we use a neural network to output one-hot vectors?

Motivation

A neural network f_{θ} is typically a continuous function.

Motivation

A neural network f_{θ} is typically a continuous function.

However, the desired output in natural language processing (a token sequence) is a discrete object.

Motivation

A neural network f_{θ} is typically a continuous function.

However, the desired output in natural language processing (a token sequence) is a discrete object.

A function that is both continuous and has a discrete range must be a constant function, which is not useful.

Motivation

A neural network f_{θ} is typically a continuous function.

However, the desired output in natural language processing (a token sequence) is a discrete object.

A function that is both continuous and has a discrete range must be a constant function, which is not useful.

Solution: Instead of outputting the token sequence itself, the neural network outputs **scores for tokens**. These scores are then converted into probabilities via the softmax function, defining a **probabilistic language model**.

Motivation

Let's formalize our terms.

Definition (Vocabulary and Token Sequences)

The set of all possible values a token can take is called the **vocabulary**, denoted by $\mathcal{V}$.

Motivation

Let's formalize our terms.

Definition (Vocabulary and Token Sequences)

The set of all possible values a token can take is called the **vocabulary**, denoted by $\mathcal{V}$.

- For a vocabulary of size D , we identify $\mathcal{V}$ with the set of integers $\{1, 2, \dots, D\}$.

Motivation

Let's formalize our terms.

Definition (Vocabulary and Token Sequences)

The set of all possible values a token can take is called the **vocabulary**, denoted by $\mathcal{V}$.

- For a vocabulary of size D , we identify $\mathcal{V}$ with the set of integers $\{1, 2, \dots, D\}$.
- The set of token sequences of length n is $\mathcal{V}^n$.

Motivation

Let's formalize our terms.

Definition (Vocabulary and Token Sequences)

The set of all possible values a token can take is called the **vocabulary**, denoted by $\mathcal{V}$.

- For a vocabulary of size D , we identify $\mathcal{V}$ with the set of integers $\{1, 2, \dots, D\}$.
- The set of token sequences of length n is $\mathcal{V}^n$.
- The set of all token sequences of any finite length is $\mathcal{V}^* = \mathcal{V}^0 \cup \mathcal{V}^1 \cup \mathcal{V}^2 \cup \dots$.

Motivation

Now we can define the central concept.

Definition (Probabilistic Language Model (Most General Form))

Let a finite vocabulary $\mathcal{V} = \{1, 2, \dots, D\}$ be fixed. A **probabilistic language model** is a function $P(\cdot|\cdot)$ that, given any finite-length token sequence $\mathbf{t} = (t_1, \dots, t_n) \in \mathcal{V}^n$, returns the conditional probability mass function of the next token.

Motivation

Now we can define the central concept.

Definition (Probabilistic Language Model (Most General Form))

Let a finite vocabulary $\mathcal{V} = \{1, 2, \dots, D\}$ be fixed. A **probabilistic language model** is a function $P(\cdot|\cdot)$ that, given any finite-length token sequence $\mathbf{t} = (t_1, \dots, t_n) \in \mathcal{V}^n$, returns the conditional probability mass function of the next token.

More formally, for any $\mathbf{t} \in \mathcal{V}^*$, the following must hold:

$$\sum_{v \in \mathcal{V}} P(v|\mathbf{t}) = 1 \tag{1}$$

Motivation

Remark

This definition is very general. It does not assume any specific model structure like Markov properties, n -grams, or Transformers [4].

Motivation

Remark

This definition is very general. It does not assume any specific model structure like Markov properties, n -grams, or Transformers [4].

It also specifies the joint distribution of an entire sequence via the chain rule of probability:

$$P((t_1, \dots, t_n)) = \prod_{i=1}^n P(t_i | (t_1, \dots, t_{i-1}))$$

Motivation

Remark

This definition is very general. It does not assume any specific model structure like Markov properties, n -grams, or Transformers [4].

It also specifies the joint distribution of an entire sequence via the chain rule of probability:

$$P((t_1, \dots, t_n)) = \prod_{i=1}^n P(t_i | (t_1, \dots, t_{i-1}))$$

Constructing the joint distribution directly is possible but harder to adapt for arbitrary-length outputs compared to modeling conditional probabilities.

Construction from a neural network

To construct a probabilistic language model from a neural network, we must solve two issues:

- **Handling Variable-Length Inputs:** How can a fixed neural network architecture process token sequences of different lengths?

Construction from a neural network

To construct a probabilistic language model from a neural network, we must solve two issues:

- **Handling Variable-Length Inputs:** How can a fixed neural network architecture process token sequences of different lengths?
- **Handling Probability Constraints:** How do we ensure the network's output is a valid probability distribution (non-negative and sums to 1)?

Handling variable-length inputs

This is addressed through special neural network architectures. Two general approaches exist:

- **Giant Neural Network Approach:** Use a huge network that accepts a very long input of fixed length L . Pad shorter inputs to match this length.

¹On the other hand, the multi-layer perceptron, which many students learn first but is not often used in practice, does not have such a recursive structure. The fact that neural networks with recursively definable structures have been successful is a manifestation of some law of nature, and its elucidation is an interesting research topic.

Handling variable-length inputs

This is addressed through special neural network architectures. Two general approaches exist:

- **Giant Neural Network Approach:** Use a huge network that accepts a very long input of fixed length L . Pad shorter inputs to match this length.
- **Variable-Length Input Neural Network Model Approach:** Use an architecture with parameter sharing and recursive structures (like RNNs or Transformers). This allows¹ constructing a network $f_{\theta}^{(n)}$ tailored to the input length n from a single set of parameters θ .

¹On the other hand, the multi-layer perceptron, which many students learn first but is not often used in practice, does not have such a recursive structure. The fact that neural networks with recursively definable structures have been successful is a manifestation of some law of nature, and its elucidation is an interesting research topic.

Handling variable-length inputs

This is addressed through special neural network architectures. Two general approaches exist:

- **Giant Neural Network Approach:** Use a huge network that accepts a very long input of fixed length L . Pad shorter inputs to match this length.
- **Variable-Length Input Neural Network Model Approach:** Use an architecture with parameter sharing and recursive structures (like RNNs or Transformers). This allows¹ constructing a network $f_{\theta}^{(n)}$ tailored to the input length n from a single set of parameters θ .

In practice, most successful models (CNNs, RNNs, Transformers) use the second approach.

¹On the other hand, the multi-layer perceptron, which many students learn first but is not often used in practice, does not have such a recursive structure. The fact that neural networks with recursively definable structures have been successful is a manifestation of some law of nature, and its elucidation is an interesting research topic.

Handling variable-length inputs

Remark (Difference between Variable-Length Input NN and an Infinite Sequence of NNs)

A crucial point is that we don't need to store an infinite number of networks $f_{\theta}^{(n)}$ for all possible lengths n .

Handling variable-length inputs

Remark (Difference between Variable-Length Input NN and an Infinite Sequence of NNs)

A crucial point is that we don't need to store an infinite number of networks $f_{\theta}^{(n)}$ for all possible lengths n .

Instead, we only need an **algorithm** that can mechanically construct the specific network $f_{\theta}^{(n)}$ on-the-fly, once the input length n is known.

Handling variable-length inputs

Remark (Difference between Variable-Length Input NN and an Infinite Sequence of NNs)

A crucial point is that we don't need to store an infinite number of networks $f_{\theta}^{(n)}$ for all possible lengths n .

Instead, we only need an **algorithm** that can mechanically construct the specific network $f_{\theta}^{(n)}$ on-the-fly, once the input length n is known.

This algorithm takes the input length, n , as an argument and dynamically constructs the appropriate network architecture for that specific length. All we need to store permanently is the fixed, finite-dimensional parameter vector θ .

Handling probability constraints

How do we ensure the output vector represents a probability mass function?

Handling probability constraints

How do we ensure the output vector represents a probability mass function?

The standard method is **softmax normalization**.

Definition (Construction via Softmax Normalization)

Given a variable-length input function $f_{\theta}^{(\cdot)}$ that maps a sequence of length n to a vector of scores in $\mathbb{R}^{|\mathcal{V}|}$, $f_{\theta}^{(n)} : \mathcal{V}^n \rightarrow \mathbb{R}^{|\mathcal{V}|}$.

Handling probability constraints

How do we ensure the output vector represents a probability mass function?

The standard method is **softmax normalization**.

Definition (Construction via Softmax Normalization)

Given a variable-length input function $f_{\theta}^{(\cdot)}$ that maps a sequence of length n to a vector of scores in $\mathbb{R}^{|\mathcal{V}|}$, $f_{\theta}^{(n)} : \mathcal{V}^n \rightarrow \mathbb{R}^{|\mathcal{V}|}$.

The probability of token v given history t is defined as:

$$P_{f_{\theta}^{(\cdot)}}(v \mid t) := \frac{\exp\left(f_{\theta,v}^{(|t|)}(t)\right)}{\sum_{u \in \mathcal{V}} \exp\left(f_{\theta,u}^{(|t|)}(t)\right)} \quad (2)$$

The resulting function $P_{f_{\theta}^{(\cdot)}}(\cdot \mid \cdot)$ is a probabilistic language model.

Handling probability constraints

In practice, we often introduce a hyperparameter called **temperature** ($T > 0$) to control the shape of the probability distribution.

Definition (Construction with Temperature)

The probability is defined by dividing the scores by T before applying softmax:

$$P_{f_{\theta}^{(\cdot)}, T}(v \mid \mathbf{t}) := \frac{\exp\left(\frac{1}{T} \cdot f_{\theta, v}^{(|\mathbf{t}|)}(\mathbf{t})\right)}{\sum_{u \in \mathcal{V}} \exp\left(\frac{1}{T} \cdot f_{\theta, u}^{(|\mathbf{t}|)}(\mathbf{t})\right)} \quad (3)$$

Handling probability constraints

Remark

Changing the temperature T ² controls the "peakedness" of the distribution.

- As $T \downarrow 0$ (low temperature): The distribution becomes sharper. Differences in scores are exaggerated, making the model more confident and deterministic. The probability mass concentrates on the highest-scoring token.

²The name "temperature" originates from statistical mechanics (the Boltzmann distribution, a probability distribution obtained by applying softmax to the product of the inverse of temperature and the negative energy corresponding to $f_{\theta,u}^{(|t|)}(t)$ in the previous definition, is fundamental in statistical mechanics).

Handling probability constraints

Remark

Changing the temperature T ² controls the "peakedness" of the distribution.

- As $T \downarrow 0$ (low temperature): The distribution becomes sharper. Differences in scores are exaggerated, making the model more confident and deterministic. The probability mass concentrates on the highest-scoring token.
- As $T \uparrow \infty$ (high temperature): The distribution becomes flatter, approaching a uniform distribution. The model becomes more random and diverse, giving even low-scoring tokens a chance.

²The name "temperature" originates from statistical mechanics (the Boltzmann distribution, a probability distribution obtained by applying softmax to the product of the inverse of temperature and the negative energy corresponding to $f_{\theta,u}^{(|t|)}(t)$ in the previous definition, is fundamental in statistical mechanics).

Handling probability constraints

Remark

Changing the temperature T ² controls the "peakedness" of the distribution.

- As $T \downarrow 0$ (low temperature): The distribution becomes sharper. Differences in scores are exaggerated, making the model more confident and deterministic. The probability mass concentrates on the highest-scoring token.
- As $T \uparrow \infty$ (high temperature): The distribution becomes flatter, approaching a uniform distribution. The model becomes more random and diverse, giving even low-scoring tokens a chance.
- When $T = 1$, we recover the original softmax definition.

²The name "temperature" originates from statistical mechanics (the Boltzmann distribution, a probability distribution obtained by applying softmax to the product of the inverse of temperature and the negative energy corresponding to $f_{\theta,u}^{(|t|)}(t)$ in the previous definition, is fundamental in statistical mechanics).

Neural network as a probabilistic language model - answer

Question: How

Neural network as a probabilistic language model - answer

Question: How do we use a neural network to output one-hot vectors?

Answer: We do not use the neural network's output directly. Instead, we regard the output as a probability distribution of the output.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How do we use a NN in discrete token generations? - Probabilistic language model

4. How do we determine the output token from a distribution? - Sampling and beam search

5. Can we adapt the generation to real application preference? - Hyperparameters

6. Summary

7. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
- 4. How do we determine the output token from a distribution? - Sampling and beam search**
5. Can we adapt the generation to real application preference? - Hyperparameters
6. Summary
7. Next Time

Question: When we regard a neural network as a probability distribution, we can sample one token.

How

Question: When we regard a neural network as a probability distribution, we can sample one token.

How can we generate a **sequence** of tokens?

Two basic ideas

A probabilistic language model $P(\cdot \mid t)$ gives us the probability for the next token.
How do we use this to generate an entire sequence?

Two basic ideas

A probabilistic language model $P(\cdot | t)$ gives us the probability for the next token.
How do we use this to generate an entire sequence?

There are two fundamental strategies.

- **Sampling (Stochastic):** At each step, randomly draw the next token according to the probability distribution $P(\cdot | t)$. This introduces diversity.

Two basic ideas

A probabilistic language model $P(\cdot | t)$ gives us the probability for the next token. How do we use this to generate an entire sequence?

There are two fundamental strategies.

- **Sampling (Stochastic):** At each step, randomly draw the next token according to the probability distribution $P(\cdot | t)$. This introduces diversity.
- **Selecting the Maximum (Deterministic):** At each step, simply pick the token with the highest probability. This leads to more consistent but potentially repetitive outputs.

Two basic ideas

Sampling requires a source of randomness.

³Note that while the Mersenne twister is excellent for most purposes, it is not suitable for cryptographic applications.

Two basic ideas

Sampling requires a source of randomness.

- True random numbers are hard to obtain on computers.

³Note that while the Mersenne twister is excellent for most purposes, it is not suitable for cryptographic applications.

Two basic ideas

Sampling requires a source of randomness.

- True random numbers are hard to obtain on computers.
- We use a **pseudo-random number generator** (PRNG).

³Note that while the Mersenne twister is excellent for most purposes, it is not suitable for cryptographic applications.

Two basic ideas

Sampling requires a source of randomness.

- True random numbers are hard to obtain on computers.
- We use a **pseudo-random number generator** (PRNG).
- A PRNG uses a **deterministic algorithm** and an initial **seed** to produce a sequence of numbers that appears random (long period, high uniformity).

³Note that while the Mersenne twister is excellent for most purposes, it is not suitable for cryptographic applications.

Two basic ideas

Sampling requires a source of randomness.

- True random numbers are hard to obtain on computers.
- We use a **pseudo-random number generator** (PRNG).
- A PRNG uses a **deterministic algorithm** and an initial **seed** to produce a sequence of numbers that appears random (long period, high uniformity).
- A famous example is the **Mersenne Twister**³ [3].

³Note that while the Mersenne twister is excellent for most purposes, it is not suitable for cryptographic applications.

Definition: pseudo-random sequences and token generators

First, let's define a pseudo-random sequence.

Definition (Pseudo-random sequence)

Given a fixed initial seed $s \in \mathbb{Z}$, a generator PRNG deterministically returns an infinite sequence of numbers in $[0, 1)$, $\mathbf{u}(s) = (u_1, u_2, \dots)$.

Definition: pseudo-random sequences and token generators

First, let's define a pseudo-random sequence.

Definition (Pseudo-random sequence)

Given a fixed initial seed $s \in \mathbb{Z}$, a generator PRNG deterministically returns an infinite sequence of numbers in $[0, 1)$, $\mathbf{u}(s) = (u_1, u_2, \dots)$.

Now, we can define a token generator.

Definition (Token Generator (General Form))

A **token generator** is a deterministic function $\text{Gen}_{(P, \text{PRNG})}$ that takes an input sequence t_{in} and a seed s , and returns an output sequence t_{out} .

$$\text{Gen}_{(P, \text{PRNG})} : \mathcal{V}^* \times \mathbb{Z} \rightarrow \mathcal{V}^* \quad (4)$$

Internally, it sequentially appends tokens based on the model P and the numbers from $\mathbf{u}(s)$, stopping when a **termination condition** is met.

Definition: pseudo-random sequences and token generators

Remark

If the initial seed s is fixed, the output is **completely deterministic**.

Definition: pseudo-random sequences and token generators

Remark

If the initial seed s is fixed, the output is **completely deterministic**.

The apparent "randomness" comes entirely from the PRNG. The generation rule itself is a deterministic mathematical function.

Sampling

This is the most basic sampling method, also known as inverse transform sampling.

Definition (Sampling using a Probabilistic Language Model)

Given an initial sequence $t^{(0)}$, a probabilistic model P , a PRNG, and a stopping condition Stop. For $i = 0, 1, 2, \dots$:

1. If $\text{Stop}(t^{(i)})$ is true, stop and return $t^{(i)}$.

Sampling

This is the most basic sampling method, also known as inverse transform sampling.

Definition (Sampling using a Probabilistic Language Model)

Given an initial sequence $t^{(0)}$, a probabilistic model P , a PRNG, and a stopping condition Stop. For $i = 0, 1, 2, \dots$:

1. If $\text{Stop}(t^{(i)})$ is true, stop and return $t^{(i)}$.
2. Calculate the cumulative distribution function (CDF):

$$F^{(i)}(v) := \sum_{u \leq v} P(u | t^{(i)}).$$

Sampling

This is the most basic sampling method, also known as inverse transform sampling.

Definition (Sampling using a Probabilistic Language Model)

Given an initial sequence $\mathbf{t}^{(0)}$, a probabilistic model P , a PRNG, and a stopping condition Stop. For $i = 0, 1, 2, \dots$:

1. If $\text{Stop}(\mathbf{t}^{(i)})$ is true, stop and return $\mathbf{t}^{(i)}$.
2. Calculate the cumulative distribution function (CDF):
$$F^{(i)}(v) := \sum_{u \leq v} P(u | \mathbf{t}^{(i)}).$$
3. Get the next random number u_{i+1} from the PRNG.

Sampling

This is the most basic sampling method, also known as inverse transform sampling.

Definition (Sampling using a Probabilistic Language Model)

Given an initial sequence $\mathbf{t}^{(0)}$, a probabilistic model P , a PRNG, and a stopping condition Stop. For $i = 0, 1, 2, \dots$:

1. If $\text{Stop}(\mathbf{t}^{(i)})$ is true, stop and return $\mathbf{t}^{(i)}$.
2. Calculate the cumulative distribution function (CDF):
$$F^{(i)}(v) := \sum_{u \leq v} P(u | \mathbf{t}^{(i)}).$$
3. Get the next random number u_{i+1} from the PRNG.
4. Find the next token: $t_{i+1} := \min\{v \in \mathcal{V} \mid F^{(i)}(v) > u_{i+1}\}.$

Sampling

This is the most basic sampling method, also known as inverse transform sampling.

Definition (Sampling using a Probabilistic Language Model)

Given an initial sequence $\mathbf{t}^{(0)}$, a probabilistic model P , a PRNG, and a stopping condition Stop . For $i = 0, 1, 2, \dots$:

1. If $\text{Stop}(\mathbf{t}^{(i)})$ is true, stop and return $\mathbf{t}^{(i)}$.
2. Calculate the cumulative distribution function (CDF):
$$F^{(i)}(v) := \sum_{u \leq v} P(u | \mathbf{t}^{(i)}).$$
3. Get the next random number u_{i+1} from the PRNG.
4. Find the next token: $t_{i+1} := \min\{v \in \mathcal{V} \mid F^{(i)}(v) > u_{i+1}\}$.
5. Append the token: $\mathbf{t}^{(i+1)} := (\mathbf{t}^{(i)}, t_{i+1})$ and repeat.

Sampling

Example (Two Steps of Naive Sampling)

Let $\mathcal{V} = \{1, 2, 3\}$, input $t_{\text{in}} = ()$, and stop at length 2. The probability distributions are:

$$P(\cdot \mid ()) = (0.50, 0.30, 0.20)$$

$$P(\cdot \mid (1)) = (0.30, 0.40, 0.30)$$

$$P(\cdot \mid (2)) = (0.10, 0.20, 0.70)$$

$$P(\cdot \mid (3)) = (0.60, 0.10, 0.30)$$

The PRNG gives us $u_1 = 0.68$, $u_2 = 0.72$. Let's find the output.

Sampling

Step 1:

- Current sequence $t^{(0)} = ()$.
- The probability distribution is $P(\cdot|()) = (0.50, 0.30, 0.20)$.
- The cumulative distribution (CDF) is $F^{(0)} = (0.50, 0.80, 1.00)$.
- Our first random number is $u_1 = 0.68$.
- We look for the smallest token v where $F^{(0)}(v) > 0.68$.
 - $F^{(0)}(1) = 0.50 \not> 0.68$
 - $F^{(0)}(2) = 0.80 > 0.68$
- So, the next token is $t_1 = 2$. New sequence $t^{(1)} = (2)$.

Sampling

Step 2:

- Current sequence $t^{(1)} = (2)$.
- The probability distribution is $P(\cdot|(2)) = (0.10, 0.20, 0.70)$.
- The CDF is $F^{(1)} = (0.10, 0.30, 1.00)$.
- Our second random number is $u_2 = 0.72$.
- We look for the smallest token v where $F^{(1)}(v) > 0.72$.
 - $F^{(1)}(1) = 0.10 \not> 0.72$
 - $F^{(1)}(2) = 0.30 \not> 0.72$
 - $F^{(1)}(3) = 1.00 > 0.72$
- So, the next token is $t_2 = 3$. New sequence $t^{(2)} = (2, 3)$.

The length is now 2, so we stop. The final output is $(2, 3)$.

Sampling

Exercise (Manual Calculation Exercise (Naive Sampling))

Let $\mathcal{V} = \{1, 2, 3\}$, start with $()$, and stop at length 2.

$$P(\cdot \mid ()) = (0.40, 0.40, 0.20)$$

$$P(\cdot \mid (1)) = (0.25, 0.25, 0.50)$$

$$P(\cdot \mid (2)) = (0.10, 0.70, 0.20)$$

$$P(\cdot \mid (3)) = (0.50, 0.30, 0.20)$$

Given $u_1 = 0.41$, $u_2 = 0.20$, find the output sequence.

Sampling

Answer

Step 1:

- $t^{(0)} = ()$.
- $P(\cdot | ()) = (0.40, 0.40, 0.20)$.
- CDF: $(0.40, 0.80, 1.00)$.
- $u_1 = 0.41$. Since $0.40 < 0.41 \leq 0.80$, we select $t_1 = 2$.
- New sequence: $t^{(1)} = (2)$.

Sampling

Answer

Step 1:

- $t^{(0)} = ()$.
- $P(\cdot | ()) = (0.40, 0.40, 0.20)$.
- CDF: $(0.40, 0.80, 1.00)$.
- $u_1 = 0.41$. Since $0.40 < 0.41 \leq 0.80$, we select $t_1 = 2$.
- New sequence: $t^{(1)} = (2)$.

Step 2:

- $t^{(1)} = (2)$.
- $P(\cdot | (2)) = (0.10, 0.70, 0.20)$.
- CDF: $(0.10, 0.80, 1.00)$.
- $u_2 = 0.20$. Since $0.10 < 0.20 \leq 0.80$, we select $t_2 = 2$.

Selecting the maximum probability element

Let's now turn to the deterministic approach. The simplest method is greedy search.

Definition (Greedy decoding)

Given an initial sequence $\mathbf{t}^{(0)}$ and a stopping condition Stop. At each step i :

1. Select the token with the highest probability:

$$t_{i+1} := \arg \max_{v \in \mathcal{V}} P \left(v \mid \mathbf{t}^{(i)} \right)$$

(Ties are broken by a fixed rule, e.g., lowest token ID).

2. Append the token: $\mathbf{t}^{(i+1)} := (\mathbf{t}^{(i)}, t_{i+1})$.
3. Repeat until Stop is met.

Selecting the maximum probability element

Example (Greedy Search (Two Steps, Numerical))

Let $\mathcal{V} = \{1, 2, 3\}$, stop at length 2. Probabilities:

$$P(\cdot \mid ()) = (0.55, 0.40, 0.05)$$

$$P(\cdot \mid (1)) = (0.20, 0.70, 0.10)$$

- **Step 1:** Given $()$, the probabilities are $(0.55, 0.40, 0.05)$. The max is at index 1. So, $t_1 = 1$.
- **Step 2:** Given (1) , the probabilities are $(0.20, 0.70, 0.10)$. The max is at index 2. So, $t_2 = 2$.

The output is $(1, 2)$.

Limitations of greedy search and beam search

Greedy search does not guarantee finding the sequence with the highest overall joint probability.

Limitations of greedy search and beam search

Greedy search does not guarantee finding the sequence with the highest overall joint probability.

The joint probability of a sequence is $P(t_1, t_2) = P(t_1 | ()) \cdot P(t_2 | (t_1))$. A locally optimal choice at step 1 might lead to a globally suboptimal sequence.

Example (Greedy Fails)

Let $\mathcal{V} = \{A, B\}$, stop at length 2.

$$P(\cdot | ()) = (0.60, 0.40)$$

$$P(\cdot | (A)) = (0.50, 0.50)$$

$$P(\cdot | (B)) = (0.95, 0.05)$$

Limitations of greedy search and beam search

Greedy Search Path:

- Step 1: $P(A|()) = 0.6 > P(B|()) = 0.4$. Choose A.
- Step 2: $P(A|(A)) = 0.5$. Choose A (tie-break).
- Result: (A, A) . Joint probability: $P(A, A) = 0.6 \times 0.5 = 0.30$.

Limitations of greedy search and beam search

Greedy Search Path:

- Step 1: $P(A|()) = 0.6 > P(B|()) = 0.4$. Choose A.
- Step 2: $P(A|(A)) = 0.5$. Choose A (tie-break).
- Result: (A, A) . Joint probability: $P(A, A) = 0.6 \times 0.5 = 0.30$.

True Best Sequence: Let's check all possibilities:

- $P(A, A) = 0.6 \times 0.5 = 0.30$
- $P(A, B) = 0.6 \times 0.5 = 0.30$
- $P(B, A) = 0.4 \times 0.95 = \mathbf{0.38}$
- $P(B, B) = 0.4 \times 0.05 = 0.02$

The sequence (B, A) has the highest probability, but greedy search missed it.

Limitations of greedy search and beam search

Beam search is an improvement over greedy search that addresses this issue.

Limitations of greedy search and beam search

Beam search is an improvement over greedy search that addresses this issue.

Idea: Instead of keeping only the single best choice at each step, keep track of the B most probable partial sequences (the "beam").

Definition (Beam Search (Beam width B))

1. Start with an empty sequence.
2. At each step i :
 - For each of the B sequences currently in the beam, generate all $|\mathcal{V}|$ possible next sequences and calculate their log-probabilities.
 - From this set of $B \times |\mathcal{V}|$ candidates, select the top B sequences with the highest overall log-probabilities. These form the new beam.
3. Repeat until a stop condition is met, then return the highest-scoring sequence from the final beam.

Limitations of greedy search and beam search

Let's re-run the previous example with beam search, using a beam width $B = 2$.

Step 1:

- Initial candidates: (A) with log-prob $\log(0.6)$ and (B) with log-prob $\log(0.4)$.
- Since $B = 2$, we keep both.
- Beam $\mathcal{B}_1 = \{(A, \log 0.6), (B, \log 0.4)\}$.

Limitations of greedy search and beam search

Let's re-run the previous example with beam search, using a beam width $B = 2$.

Step 1:

- Initial candidates: (A) with log-prob $\log(0.6)$ and (B) with log-prob $\log(0.4)$.
- Since $B = 2$, we keep both.
- Beam $\mathcal{B}_1 = \{(A, \log 0.6), (B, \log 0.4)\}$.

Step 2:

- Expand from A : (A, A) with log-prob $\log(0.6) + \log(0.5) = \log(0.3)$. And (A, B) also with log-prob $\log(0.3)$.
- Expand from B : (B, A) with log-prob $\log(0.4) + \log(0.95) = \log(0.38)$. And (B, B) with log-prob $\log(0.02)$.
- Candidates' log-probs: $\{\log 0.30, \log 0.30, \log 0.38, \log 0.02\}$.
- The top 2 are (B, A) and (A, A) (or (A, B)). The best is (B, A) .

Generating a sequence - answer

Question: How can we generate a **sequence** of tokens?

Answer:

Generating a sequence - answer

Question: How can we generate a **sequence** of tokens?

Answer: We iteratively select the maximum probability token or stochastically sample a next token according to the probability distribution determined by the neural network.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
- 4. How do we determine the output token from a distribution? - Sampling and beam search**
5. Can we adapt the generation to real application preference? - Hyperparameters
6. Summary
7. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
4. How do we determine the output token from a distribution? - Sampling and beam search
- 5. Can we adapt the generation to real application preference? - Hyperparameters**
6. Summary
7. Next Time

Question: Since we do not directly use the neural network to determine the output, we have room for adapting the token generation rule so that it is preferable in applications.

What

Question: Since we do not directly use the neural network to determine the output, we have room for adapting the token generation rule so that it is preferable in applications.

What modifications we can apply to the token generation algorithm, without changing the neural network itself?

Modifying probabilistic language models

In practice, the raw probability distribution from the model is often modified before sampling.

Modifying probabilistic language models

In practice, the raw probability distribution from the model is often modified before sampling.

Motivations:

- **Preventing Repetition:** Avoid generating monotonous, repetitive text.

Modifying probabilistic language models

In practice, the raw probability distribution from the model is often modified before sampling.

Motivations:

- **Preventing Repetition:** Avoid generating monotonous, repetitive text.
- **Excluding Low-Score Tokens:** Avoid generating nonsensical text by completely ruling out tokens that the model considers very unlikely.

Modifying probabilistic language models

Here are some popular techniques:

- **Repetition Penalty [2]:** Decrease the scores of tokens that have already appeared in the generated sequence.

Modifying probabilistic language models

Here are some popular techniques:

- **Repetition Penalty [2]:** Decrease the scores of tokens that have already appeared in the generated sequence.
- **Temperature:** As we've seen, lowering $T < 1$ makes the distribution sharper and less random.

Modifying probabilistic language models

Here are some popular techniques:

- **Repetition Penalty [2]:** Decrease the scores of tokens that have already appeared in the generated sequence.
- **Temperature:** As we've seen, lowering $T < 1$ makes the distribution sharper and less random.
- **Top-k Sampling:** Consider only the top k most probable tokens and set the probability of all others to zero.

Modifying probabilistic language models

Here are some popular techniques:

- **Repetition Penalty [2]:** Decrease the scores of tokens that have already appeared in the generated sequence.
- **Temperature:** As we've seen, lowering $T < 1$ makes the distribution sharper and less random.
- **Top-k Sampling:** Consider only the top k most probable tokens and set the probability of all others to zero.
- **Top-p (Nucleus) Sampling [1]:** Consider the smallest set of most probable tokens whose cumulative probability exceeds a threshold p . This set is called the "nucleus."

Modifying probabilistic language models

These modifications are often applied in a specific order. Let's look at the common pipeline used by libraries like Hugging Face.

Definition (Construction of Conditional PMF in Hugging Face)

Given scores $s_{|t}$ for history t :

1. Apply **repetition penalty**.
2. Apply **temperature** scaling.
3. Apply **top- k** filtering.
4. Apply **softmax** to get probabilities $\pi(v)$.
5. Apply **top- p** (nucleus) filtering and renormalize.

Modifying probabilistic language models

Let's look closer at the repetition penalty step.

1. (**repetition penalty**) Let $H(t)$ be the set of previously seen tokens and $\lambda > 0$ be the penalty factor.

$$s_{v|t}^{\text{rep}} \leftarrow \begin{cases} s_{v|t} & \text{if } v \notin H(t), \\ \frac{1}{\lambda} s_{v|t} & \text{if } v \in H(t) \text{ and } s_{v|t} \geq 0, \\ \lambda s_{v|t} & \text{if } v \in H(t) \text{ and } s_{v|t} < 0. \end{cases}$$

Modifying probabilistic language models

Example (Hugging Face Style Sampling)

Let $\mathcal{V} = \{1, 2, 3, 4\}$, start with $()$, and stop at length 2. Scores:

$$\mathbf{s}_{|()} = (2.0, 1.0, 0.0, -1.0)$$

$$\mathbf{s}_{|(1)} = (1.5, 0.0, 0.5, -0.2)$$

Hyperparameters: $T = 0.5$, $\lambda = 1.2$, $k = 3$, $p = 0.9$. PRNG gives $u_1 = 0.50$, $u_2 = 0.90$. Find the output.

Modifying probabilistic language models

Step 1 (History $t = ()$, Seen tokens $H(()) = \{\}$)

1. **(rep)** No tokens seen, scores unchanged: $(2.0, 1.0, 0.0, -1.0)$.
2. **(temp)** Divide by $T = 0.5$ (i.e., multiply by 2): $(4.0, 2.0, 0.0, -2.0)$.
3. **(top- k)** Keep top $k = 3$ scores. 4th score becomes $-\infty$: $(4.0, 2.0, 0.0, -\infty)$.
4. **(softmax)** Probabilities $\pi \approx (0.8668, 0.1173, 0.0159, 0)$.
5. **(top- p)** Cumulative sum is $(0.8668, 0.9841, \dots)$. For $p = 0.9$, nucleus is $\{1, 2\}$.
Renormalize: $\tilde{\pi} \approx (0.8808, 0.1192, 0, 0)$.
6. **(sample)** CDF is $(0.8808, 1.0)$. $u_1 = 0.50 < 0.8808 \implies t_1 = 1$.

Modifying probabilistic language models

Step 2 (History $t = (1)$, Seen tokens $H((1)) = \{1\}$)

1. **(rep)** Score for token 1: $1.5/\lambda = 1.5/1.2 = 1.25$. Scores: $(1.25, 0.0, 0.5, -0.2)$.
2. **(temp)** Divide by $T = 0.5$: $(2.5, 0.0, 1.0, -0.4)$.
3. **(top- k)** Keep top $k = 3$ scores: $(2.5, 0.0, 1.0, -\infty)$.
4. **(softmax)** Probabilities $\pi \approx (0.7662, 0.0629, 0.1710, 0)$.
5. **(top- p)** Sorted probs: $(0.7662_1, 0.1710_3, 0.0629_2)$. Cum sum: $(0.7662, 0.9372, \dots)$. Nucleus is $\{1, 3\}$. Renormalize: $\tilde{\pi} \approx (0.8176, 0, 0.1824, 0)$.
6. **(sample)** CDF is $(0.8176, 0.8176, 1.0, 1.0)$. $u_2 = 0.90$. Since $0.8176 < 0.90 \leq 1.0$, we select $t_2 = 3$.

Final output: $(1, 3)$.

Sampling hyperparameters - answer

Question: What

Sampling hyperparameters - answer

Question: What modifications we can apply to the token generation algorithm, without changing the neural network itself?

Answer:

Sampling hyperparameters - answer

Question: What modifications we can apply to the token generation algorithm, without changing the neural network itself?

Answer: To increase the diversity in generation, we can modify the generation algorithm by changing, e.g., the repetition penalty, temperature, Top-k, Top-p, etc.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
4. How do we determine the output token from a distribution? - Sampling and beam search
- 5. Can we adapt the generation to real application preference? - Hyperparameters**
6. Summary
7. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
4. How do we determine the output token from a distribution? - Sampling and beam search
5. Can we adapt the generation to real application preference? - Hyperparameters
- 6. Summary**
7. Next Time

Summary

Let's summarize the key takeaways from today's lecture.

- Neural networks, being continuous functions, are naturally used as **probabilistic language models**. They output scores, which are converted to a probability mass function (PMF) via softmax. The joint probability of a sequence is then defined by the chain rule.

Summary

Let's summarize the key takeaways from today's lecture.

- Neural networks, being continuous functions, are naturally used as **probabilistic language models**. They output scores, which are converted to a probability mass function (PMF) via softmax. The joint probability of a sequence is then defined by the chain rule.
- We can rigorously define various text generation methods within a unified framework of a **token generator**, which is a deterministic function of a model, a stopping condition, and a seed for a pseudo-random number generator.

Summary

Let's summarize the key takeaways from today's lecture.

- Neural networks, being continuous functions, are naturally used as **probabilistic language models**. They output scores, which are converted to a probability mass function (PMF) via softmax. The joint probability of a sequence is then defined by the chain rule.
- We can rigorously define various text generation methods within a unified framework of a **token generator**, which is a deterministic function of a model, a stopping condition, and a seed for a pseudo-random number generator.
- This framework includes methods like greedy search, beam search, and various sampling techniques that modify the probability distribution using temperature, top- k , top- p , and repetition penalty.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
4. How do we determine the output token from a distribution? - Sampling and beam search
5. Can we adapt the generation to real application preference? - Hyperparameters
- 6. Summary**
7. Next Time

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How do we use a NN in discrete token generations? - Probabilistic language model
4. How do we determine the output token from a distribution? - Sampling and beam search
5. Can we adapt the generation to real application preference? - Hyperparameters
6. Summary

7. Next Time

Next time

Next time, we will discuss the **embedding layer**.

Next time

Next time, we will discuss the **embedding layer**.

This is the initial layer in many probabilistic language models. We will focus on:

- How it maps discrete tokens from the vocabulary into a continuous vector space.

Next time

Next time, we will discuss the **embedding layer**.

This is the initial layer in many probabilistic language models. We will focus on:

- How it maps discrete tokens from the vocabulary into a continuous vector space.
- The learning rules for this mapping.

Next time

Next time, we will discuss the **embedding layer**.

This is the initial layer in many probabilistic language models. We will focus on:

- How it maps discrete tokens from the vocabulary into a continuous vector space.
- The learning rules for this mapping.
- Its statistical interpretation.

References i

- [1] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi.
The curious case of neural text degeneration.
In Proceedings of ICLR 2020, 2020.
- [2] Nitish Shirish Keskar, Bryan McCann, Lav R Varshney, Caiming Xiong, and Richard Socher.
Ctrl: A conditional transformer language model for controllable generation.
arXiv preprint arXiv:1909.05858, 2019.

References ii

- [3] Makoto Matsumoto and Takuji Nishimura.
Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator.
ACM Transactions on Modeling and Computer Simulation (TOMACS), 8(1):3–30, 1998.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.
In Proceedings of NeurIPS 2017, 2017.